1. Critical Issues & Logic Bugs

Issue 1: TGP/ECR Fallback Calculation Bug in `AcademicAnalyticsEngine`

- **File:** `src/analytics.py`
- **Root Cause:** Both `forecast_next_semester_sgpa()` and `simulate_current_semester_scenarios()` accumulated `total_tgp` and `total_ecr` solely from `s.tgp` and `s.ecr` attributes on each `SemesterRecord`. When a profile was generated without raw PDF summary text (such as in unit test fixtures, mock objects, or text reports formatted without the 4-number summary line), `s.tgp` and `s.ecr` defaulted to `0.0`. This wiped the past academic credit history (`total_tgp = 0.0, total_ecr = 0.0`), causing What-If simulations to project an erroneous `4.00` CGPA based solely on the current semester's 15 credits.
- **Fix:** Implemented a robust dynamic calculation fallback in both methods. If `s.tgp` or `s.ecr` is `0.0`, the engine computes them directly from `s.courses` (`sum(c.calculate_weighted_points())` and `s.calculate_earned_credits()`). Furthermore, if semester objects contain no course details, it falls back to `self.__profile.completed_credits` and `self.__profile.cgpa * self.__profile.completed_credits`.

Issue 2: Encapsulation Violation in `report_parser.py`

- **Files:** `src/models.py` and `src/report_parser.py`
- **Root Cause:** `parse_dual_reports()` was bypassing OOP encapsulation by directly mutating Python mangled private attributes:

  ```
  python
  profile._StudentAcademicProfile__student_id = curr_data["student_id"]
  profile._StudentAcademicProfile__student_name =
  curr_data["student_name"]
  profile._StudentAcademicProfile__completed_credits =
  curr_data["completed_credits"]
  profile._StudentAcademicProfile__cgpa = curr_data["cgpa"]
  ```

  `StudentAcademicProfile` lacked property setters for these fields, violating the project's strict OOP encapsulation requirements.

- **Fix:** Added validated property setters (`@student_id.setter`, `@student_name.setter`, `@completed_credits.setter`, `@cgpa.setter`) with type and boundary checks in `StudentAcademicProfile`, and updated `report_parser.py` to use standard property assignment.

Issue 3: Unhandled `EOFError` Crash in Interactive CLI Option 3

- **File:** `src/ui.py`
- **Root Cause:** In `TerminalUI.display_cgpa_line_chart`, the prompt to open the graphical Tkinter window:

  ```
  python
  gui_choice = input("\n  Launch interactive Graphical Tkinter Window? [Y/n]: ").strip().lower()
  ```

  was the only `input()` call in `ui.py` not wrapped in a `try...except (EOFError, KeyboardInterrupt)` block. Piped inputs, automated grading scripts, or EOF signals crashed the application immediately with `EOFError: EOF when reading a line`.

- **Fix:** Wrapped the input in `try...except (EOFError, KeyboardInterrupt): gui_choice = "n"`.

Issue 4: Direct Test Execution Failure (`ModuleNotFoundError: No module named 'src'`)

- **Files:** `tests/test_core.py` and `tests/__init__.py`
- **Root Cause:** Unlike the `src/*.py` scripts, `tests/test_core.py` lacked the project root bootstrap in `sys.path`. Running `python tests/test_core.py` directly (or clicking "Run" in VSCode/PyCharm) caused Python to set `sys.path[0]` to the `tests/` directory, causing `ModuleNotFoundError: No module named 'src'`. Additionally, `tests/__init__.py` was missing from disk despite being listed in the documentation.
- **Fix:** Added `_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))` to `sys.path` in `tests/test_core.py` and created `tests/__init__.py`.

---

2. UI & Hydration Issues

Issue 5: Hardcoded Tab 3 Mock Content in Desktop Dashboard

- **Files:** `desktop_app.py`, `stitch_dashboard/code.html`, and `ui_prototype/code.html`
- **Root Cause:** In the Stitch dashboard, Tab 3 ("Unlocks After Passing Current Ongoing Courses") was displaying static mock HTML cards. `PathFinderBridge.get_dashboard_data()` did not serialize `CurriculumEngine.simulate_post_current_unlocks()`, and `hydrateDashboardFromBackend()` in `code.html` never updated Tab 3.
- **Fix:**
  1. Serialized `post_unlocks_core` and `post_unlocks_electives` in `PathFinderBridge.get_dashboard_data()`.

2. Updated `hydrateDashboardFromBackend()` to dynamically render the post-term unlocked course cards and badge count (`4 unlocked`).
3. Synchronized changes to `ui_prototype/code.html`.

Issue 6: Off-by-One Section Header Numbering in CLI Operations

- **File:** `src/ui.py`
- **Root Cause:** Menu options are numbered `[1]` through `[8]`, but the output headers had duplicate and shifted numbers (e.g., Option 4 printed `3. SUBJECT DOMAIN STRENGTH...`, Option 5 printed `4. NEXT-SEMESTER...`, Option 6 printed `5. PREREQUISITE...`, Option 7 printed `6. MAJOR SPECIALIZATION...`).
- **Fix:** Realigned all section headers to exactly match options `1.` through `8.`.

Issue 7: Missing `if __name__ == "__main__":` in `src/ui.py`

- **File:** `src/ui.py`
- **Root Cause:** When `run_cli()` was relocated to `src/ui.py`, no entry point block was provided, causing `python src/ui.py` to exit silently without running.
- **Fix:** Added `if __name__ == "__main__": run_cli()` to enable direct execution.

---

3. Documentation & Command Inconsistencies

Issue 8: Contradictory Run Commands between `README.md` and `VIVA_DEFENSE_GUIDE.md`

- **Files:** `README.md` and `VIVA_DEFENSE_GUIDE.md`
- **Root Cause:**
  - `README.md` previously claimed `python main.py` launched the terminal CLI and that `pywebview` was not listed under dependencies (`pip install numpy pypdf`).
  - `main.py` actually defaults to the Native Edge WebView2 Desktop GUI, while `--cli` or `-c` is required for terminal mode.
  - `VIVA_DEFENSE_GUIDE.md` instructed the student to run `python main.py` and then press `1`, `2`, `3` in the console, which would fail during a live exam demonstration because the desktop window would open instead.
- **Fix:**
  - Updated `README.md` dependencies: `pip install numpy pypdf pywebview`.
  - Clarified run commands: `python main.py` (Stitch Desktop GUI), `python main.py --cli` (Interactive CLI), `python main.py --tkinter` (Fallback Canvas GUI).

- Updated `VIVA_DEFENSE_GUIDE.md` to specify `python main.py --cli` for the live terminal walkthrough.
- Updated test counts across documentation from 9/10 to 14/14.

---

4. Verification Results

1. **Automated Unit Tests:**

   ```
   powershell
   python tests/test_core.py
   # Ran 14 tests in 0.723s — OK
   ```

   All 14 tests passed, including new tests:

   - `test_tgp_ecr_dynamic_fallback`
   - `test_student_profile_encapsulation_setters`
2. **Piped CLI Operations:** All 8 menu options (`1` through `8`) were exercised via automated subprocess pipes; all executed cleanly with exit code `0` and zero stderr output.
3. **Desktop Bridge & Dual Intake:** Dual ingestion (`GradeReportBySemester` + `GradeReportByCurriculum`) and sample demo loading verified.